

BMB 306

YAZILIM MÜHENDİSLİĞİ

Kaliteli Kodlama

∞ Ayarları Ne Kadar Kurulabilir Bir Program (Configurability)

- Eski zamanlarda programlar konfigürasyona açık değildi. Programın kullandığı değerler programın içinde kalıcı olarak ayarlanmıştı (yani çakılmıştı). Bu değerlerden kastımız örneğin maksimum voltaj, hafıza limiti, minimum girilebilecek değer, maksimum bekleme süresi gibi değerlerdir.
- Yakın zamanlarda bu durumu düzeltmek için “Değişikliğe kapalı ama eklemeye açık programla” ilkesinden de güç alarak yazılımcılar programlarındaki her parametreyi ayarlanabilir olarak kullanıcıya sunmaya başladılar. Ya ayar pencereleri ile ya da ayar dosyaları ile bu imkân kullanıcıya sağlandı. Bu değişim önceleri iyi oldu, değiştirilmesi neredeyse imkânsız olan eski tip programlardan bizi kurtardı.
- Bu durumun da dezavantajlarının olduğunu ortaya çıkması gecikmedi. Yeni programlar artık o kadar çok ayar imkânı sağlıyordu ki bu labirent gibi ayar pencereleri veya ayar dosyalarında kaybolup gidiyordunuz. Ayarlar bir bozuldu mu düzeltene kadar canınız çıkıyordu. Üstelik çoğu zaman ayar dosyaları başlangıç değerleri ile geldiği için kendinize has bir ayar yapmak için saatlerce uğraşmanız gerekebiliyordu.
- Günümüzde size önereceğimiz yöntem şudur: “Programınızı yeterince ayar yapılabilir üretin, ne eksik ne de fazla.” Eğer bazı önemli özelliklere ayar imkânı vermezseniz kullanıcınız programdan verim alamaz ve zarar görür, eğer çok fazla özelliğe ayar imkânı verirsiniz kullanıcınız ayarları yapamaz ve yine zarar görür. Kullanıcının ayar yapmasının anlamı olmayan veya ayarlamakta güçlük çekeceği özellikleri kendiniz ayarlı olarak kullanıcıya sunun. Hatta mümkünse programınız kendini işletim sistemi veya donanıma göre otomatik olarak ayarlayabilsin.

Kaliteli Kodlama

- ☞ Ne Zaman Assembly? Ne Zaman Yüksek Seviyeli Dil? Ne Zaman Nesne Tabanlı Programlama?
 - Pratik olarak edindiğimiz tecrübe şudur:
 - 0 -10,000 satır arasında büyüklükteki kod Assembly dilinde yazılabilir (Elbette isterseniz yüksek seviyeli bir dil de kullanabilirsiniz),
 - 10,000-100,000 satır arasında yüksek seviyeli bir dil kullanmanızı tavsiye ederiz,
 - 100,000 satırdan daha büyük sistemlerde mümkün olduğunca yüksek seviyeli ve nesne tabanlı (object oriented) bir dil kullanmalısınız.
 - Yukarıdaki üçüncü madde için notumuz şudur. Eğer hız gerçekten sorun ise yüksek seviyeli ancak nesne tabanlı olmayan bir dil kullanınız. Eğer hız probleminiz yoksa ise tereddüt etmeden Nesne Tabanlı bir dil kullanınız.
 - Benzer şekilde birinci madde için yorumumuz şudur. Eğer hız ve yer probleminiz varsa kısmen veya tamamen Assembly dilini kullanabilirsiniz. Ancak böyle bir problem yok ise tereddüt etmeden Yüksek Seviyeli bir dil kullanınız.

Kaliteli Kodlama

- ✎ Yazdığınız Her Modülün Bir "Sanal Olarak Uygula" İşlevi Olmalıdır
 - Diyelim ki on tane sayıyı küçükten büyüğe doğru sıralayan basit bir modülünüz olsun.
 - Bu modüle denemeGirdisiVer isimli bir işlev koymayı unutmayınız. Örneğimizde bu işlev ile rastgele 10 sayı seçip bunlar ile modülün sıralama işlevini çağırabilmelisiniz.
 - Aslında bu örnekte bir test programı yazarak da bu işi halledebiliriz.
 - Ancak modüllere her zaman test programından girdi vermek mümkün olmayabilir.
 - Örneğin dışarıdan bir TCP/IP mesajı alan bir modül düşünelim; donanım olarak bu modüle uygun bir TCP/IP mesajı göndermek mümkün olmayabilir.
 - Bu durumda test programı ile mesaj enjekte etmek imkânı olmayacaktır.
 - Oysa ki, modülümüzün içine gömülü olan “SanalBirMesajUygula()” şeklinde bir fonksiyon olursa, bu sayede test programından bu fonksiyon çağrılarak, modülümüzü sanki gerçek bir mesaj gelmiş gibi, kolayca test edebiliriz.

Kaliteli Kodlama

☞ Üç Kez Tekrar Etmeme Kuralı (Rule Of Three)

- Yeni başlayanlara aynı kodu tekrar etmeyin diye sık sık tavsiyelerde bulunulur. Bu haklı bir tavsiyedir. Angaryayı engellemek yönü yanında çok daha önemli bir sebebi vardır.
- Kodda bir hata çıktığında genelde bir yerde düzeltilirken diğer yerde düzeltmeyi yapmak unutulur.
- Bu sebeple programımızda aynı kod iki ayrı yerde gerekiyorsa bir fonksiyon içine koymak gayet mantıklıdır.
- Buraya kadar tamam ancak bunun sınırı nedir.
- Her tekrar eden kodu bir fonksiyona mı alacağız?
 - Eğer bir kod iki yerde geçiyorsa seçim size kalmıştır, ister fonksiyon içine alırsınız ister almazsınız.
 - Ancak eğer aynı kod programınızda en az üç yerde birden geçiyorsa mümkünse lütfen bir fonksiyon içine alın.

Kaliteli Kodlama

- ☞ Başkalarının Kodunu Okuyun ve Başka Programcılarla Konuşun
 - İyi bir programcı olmak için kod yazmak kadar başkalarının yazdıkları kodları da okuyup incelemek zorundasınız.
 - Bu kodlar gerek aynı projedeki kodlar olsun gerekse internetteki kodlar olsun size yeni bakış açıları kazandıracaktır.
 - Bunun yanısıra başka programcılar ile konuşup fikir alışverişinde bulunmak da size yeni ufuklar açacaktır.
 - Aynı şekilde siz de kodlarınızı başkalarına okutup fikirlerini almalısınız.

Kaliteli Kodlama

∞ Edinebildiğiniz En İyi ve Kendinizi Rahat Hissettiğiniz Bir Editörü Kullanın

- Yazılım geliştirmek için bir IDE'ye yani yazılım geliştirme ortamına veya kısaca bir editöre ihtiyacınız var.
- Bu mümkünse piyasadaki en iyi ve en çok tavsiye edilen editör olsun.

Kaliteli Kodlama

∞ Ekran Seçimi

- Yazılım geliştirmek için uygun ekran nedir diye düşünürsek; koddaki fonksiyonlarınızı daha rahat görebileceğiniz kadar boyu büyük ve iki kod dosyasını karşılaştırabilecek kadar eni büyük bir editör seçmeye özen gösterin.
- Yani ekranın hem eni hem de boyu makul oranda büyük olmalı.
- Çok geniş fakat kısa ekranlar sinema seyretmek için iyi fakat yazılım yazmak için uygun değildir.
- Aynı şekilde çok yüksek ekranlar da yazı yazmak için iyi ama iki yazılım dosyasını karşılaştırmak için iyi değildir.
- Bu sırada 22 inch ve daha büyük monitörler bu özellikleri sağladığı için yazılımcıların gözdesi olmuş durumdadır.

Kaliteli Kodlama

∞ Veritabanına Her Şeyi Koymayın

- Veritabanı kullanmak bilgilerinize daha hızlı ulaşmak için iyi bir yöntemdir. Ancak veritabanının hızı fiziksel olarak hızlı olduğundan değil kullandığı algoritmalarından kaynaklanır.
- Aslında aynı bilgiyi bir dosyaya ve bir de veritabanına koyduğunuzu varsayarsak dosyadaki bilgiye erişmek veritabanındaki bilgiye erişmekten daha hızlıdır.
- Ancak arama yapmak, filtreler oluşturmak söz konusu olduğunda dosyadan istenilen bilgiye erişim çok uzun zaman alabilir.
- Veritabanı programında gömülü olan ve yılların tecrübesi ile akıllıca yazılmış algoritmalar ise bu işi çok kısa zamanda halleder. Bu bilgiler ışığında şu tavsiyeye kulak verebiliriz:
- Veritabanına bilgi sisteminizdeki tüm bilgileri koymayın, sadece aranacak, filtrelenecek (veya aranma, filtrelenebilir ihtimali olan) bilgileri koyun.

Kaliteli Kodlama

∞ Veritabanına Her Şeyi Koymayın (devam)

- Örneğin bir personel listesi tutacaksanız veritabanına personelin adını, sicilini, telefonunu koymak gayet iyi bir fikirdir, ancak personelin fotoğraf dosyalarını veritabanına koymayınız.
- Bu fotoğraf dosyalarını çok iyi bir isimlendirme uygulayarak sabit diskte güvenilir bir yerde tutunuz.
- Resim, müzik, vb dosyaları veritabanına koymak çoğu zaman veritabanını hantallaştırmaktan başka bir işe yaramaz.
- Şimdi de daha zor bir örnek verelim; Eğer söz konusu olan personelin özgeçmiş bilgileri ise ne yapmalı.
- Veritabanımı tercih etmeli yoksa dosya olarak mı saklamalı.
- Kendinize bu durumda şu soruyu sormalısınız; “Bu bilgiler arama veya filtrelemede kullanılacak mı veya ileride kullanılma ihtimalleri var mı?”.
- Eğer cevabınız “Evet” ise özgeçmiş bilgilerini veritabanına koyun.
- Eğer cevabınız “Hayır” ise o zaman özgeçmişleri sabit diskte tutabilirsiniz.
- Şunu tekrar etmeyi uygun buluyorum; eğer dosyaları diskte tutarsanız ulaşımın kolay ve tutarlı olması için çok düzenli bir isimlendirme geleneği uygulamayı unutmayın. Örneğin “KimlikNo” veya “AdSoyadDogumTarihiAyiriciNo” gibi.

Kaliteli Kodlama

🌀 Aynı Değeri İki Ayrı Yerde Saklamayın

- İngilizce Single Source of Truth (SSOT) ya da Türkçe adı ile “Gerçeğe tek kaynaktan ulaşılmalı” prensibi diyebileceğimiz bu kavram programcılığın tüm alanlarında ama özellikle veritabanı uygulamalarında çok önemlidir.
- Örnekle açıklamak gerekirse bir değeri iki ayrı değişkende saklamak karışıklığa yol açar ve bu karışıklık “Gerçeğe tek kaynaktan ulaşılmalı” prensibine uyulmadığından kaynaklanır.
- Aynı şekilde veritabanında aynı değer için birden fazla kayıt tutmak da karışıklığa yol açar. Özellikle silme değiştirme işlemlerinde kayıtlardan birisi işlenip diğerinin işlenmesi unutulursa veya sorun olursa ilgili değer tutarsız hale gelir.
- Veritabanı uygulamalarında normalizasyon işleminin amaçlarından biri de “Gerçeğe tek kaynaktan ulaşılmalı” prensibi doğrultusunda tabloları düzenlemeye dayanır.
- Normalizasyon işlemi ile tablolarda tekrarlar engellenir. Gerçekten geçerli aksi yönde bir sebebiniz yok ise veritabanı tablolarınızı normalize ediniz, normalizasyon için vereceğiniz ufak bir emeğin karşılığını fazlasıyla alacaksınız.

Kaliteli Kodlama

☞ Kötü Kedi "goto" / Program Akışında Zor Virajlar

- Goto sözcüğü, bazı programlama dillerinde program akışında bir noktadan diğer bir noktaya direk atlamaya yarayan bir anahtar sözcüktür. Programcılığa başlayan herkes kısa bir zaman sonra “goto” kelimesinin ne kadar kötü, sakınılması gereken, “tüh kaka” birşey olduğunu duyup bunu kabullenir. Gerçekten de “goto” kelimesi yapısal programlamayı bozduğu için pek tercih edilmez. Üstelik teorik olarak “goto” kullanmadan aynı programı yazmak mümkündür.
- Peki hiç mi yararı yok bu “goto’nun. Hiç yararı yoksa programa dillerine neden koyuyorlar. Aslında dünyadaki her şeyin zararı ve yararı olduğu gerçeği gibi “goto’nun da yararlı olduğu yerler vardır. Bu anahtar sözcüğünü aşağıdaki iki durum için hoş görebiliriz:
 - Bir işlev içinden, hata oluştuğunda hata kontrolü (error handling) yaparak çıkmak için işlevin en sonundaki hata kontrolü koduna gitmek için “goto” kullanılabilir.
 - Çok derin, iç içe if bloklarından (genelde bir hata olduğunda) ani olarak kurtulabilmek için goto kullanılabilir.
- Tekrar etmek gerekirse bu her iki durum için de aslında “goto” kullanmadan çözüm mümkündür, ancak bu çözüm kodu karmaşıktır ve okunaklılığı büyük ölçüde baltalıyorsa alternatif olarak “goto’yu kullanabiliriz.

Kaliteli Kodlama

🌀 Kodum Release Modunda Çalışmıyor Ama Debug'da Çalışıyor; Nasıl Olur?

- Bu sorunun sebebi çoğu kişinin ilk etapta suçladığı gibi derleyicinin release ve debug modlarında farklı davranan bozuk bir derleyici olması değildir. Genelde bu sorunun iki adet cevabı olur:
 - Zamanlama hatası: Program çalıştığında debug ve release modunda tüm işlemlerin aynı sırada yapıla-cağı diye bir kural yoktur. Özellikle de modern derleyiciler kodu o kadar optimize ederler ki neredeyse hangi çok çekirdekli (multi threaded) bir uygulamada hangi çekirdeğin önce hangisinin sonra çalışacağını tahmin etmek çok zordur. Üstelik bazen bir çekirdek daha henüz diğer çekirdek hazır olmadan hatta daha yaratılmadan çalışıp sistemi kolayca çökertebilir. Bunun için kodunuza koruma önlemleri koymalısınız.
 - İklendirilmeyen Değişken: Derleyiciler debug modunda değişkenlere ilk değer atama ve onları sıfırlama konusunda daha titiz davranırlar. Örneğin bir boolean değişkeniniz olsun. Derleyici debug modunda bunun ilk değerini False değerine (yani genelde sıfıra) çekerken, release modunda bu değişkene hiç bir değer atamayabilir ve değişkenin değeri şansınıza göre bazen True bazen de False olur. Bu da bize programımızın debug ve release modunda farklı davranmasını çok güzel açıklar.

Kaliteli Kodlama

- 🌀 Test Adımlarım Bir Kalıyor, Bir Geçiyor, Hadi İşin İçinden Çık
 - Genelde bu sorunun iki adet cevabı olur:
 - Zamanlama hatası: Test programınız çalıştığında test adımlarının sırasıyla yapıldığını zannedersiniz. Ancak çoğu zaman bu böyle değildir. Test programları yüksek seviyeli programlar oldukları için sizin ekranda sıralı çalıştığını zannettiğiniz testler altyapı tarafından farklı sıralarda çalıştırılıyor olabilir. Ayrıca test adımlarının bazıları diğerlerinden uzun sürdüğü için yine sıra bozuluyor olabilir. Bunun için test adımlarınız arasında bekleme süreleri koymak, önkoşul koymak ve benzeri önlemleri almalısınız.
 - İklendirilme sorunu: Koşturduğunuz test ilk seferde kalıyor ve diğer seferlerde geçiyorsa sorun iklendirme sorunu olabilir. İlk test sırasında daha ön ayarlar yapılmadığı için test kalıyor olabilir. İkinci koşuşta ise farkında olmadan o ayarlar testin başarılı olarak geçebileceği koşula geliyordur.

Kaliteli Kodlama

☞ Hata Yapmam Demeyin, Kod İçerisinde Birçok Şeyi Hatta Kendinizi Bile Test Edin

- Kodunuzu yazdığınızda hata kontrol kodları koymayı unutmayın. Defansif kodlama da denilen bu tarz kodlamaya en iyi açıklamanın bir örnekle olacağını düşündük.

Hata kontrolü koymadan önce:

```
void yeniüyeEkle(short yeniüyeSayisi)
{
    toplamüyeSayisi += yeniüyeSayisi;
    uyeSayisiniGoruntule ();
}
```

Hata kontrolü koyduktan sonra:

```
void yeniüyeEkle(short yeniüyeSayisi)
{
    if (yeniüyeSayisi < 0)
    {
        // Hata mesajı göster
        return;
    }
    toplamüyeSayisi += yeniüyeSayisi;
    if (toplamüyeSayisi > 0)
        uyeSayisiniGoruntule();
}
```

Kaliteli Kodlama

☞ Sadece Tek Bir Sorumluluk Prensibi

- Bir yazılım parçacığı sadece ve sadece bir işi tam ve iyi yapmalıdır. Bu yazılım parçacığı bir işlev, prosedür veya nesne tabanlı programlamadaki bir sınıf olabilir. Bu kurala uymak bakımı yapılabilir, tekrar kullanılabilir ve kolayca test edilebilir kod yazmak için çok gereklidir.
- Örnek vermek gerekirse diyelim ki bu kurala uymadınız ve bir işlevde iki tane işi yapıyorsunuz. Eğer birinci işin algoritması değişirse bu işlevin her iki görevini de kullanan bütün yerleri yeniden test etmeniz gerekecektir. Oysa ki bu iki işi iki adet farklı işleve koymuş olsaydık sadece birinci işlevi kullanan yerleri yeniden test etmemiz gerekecekti.
- Aslında kural çok basit bir kural olduğu için bu kuralı hemen hemen yazılım yapan herkes bilir. Sorun kuralı ihlal edip hataya düşmenin sıklığındadır. En deneyimli yazılımcı bile bir üniteye birden fazla iş koyma hatasına düşer.
- Her kuralda olduğu gibi bu prensibin de suistimal edilmesi kötü sonuçlar doğurur. Örneğin deneyimsiz bir programcı basit bir iş parçasını o kadar çok alt parçalara ayırır ki kod okunaklılığını yitirir ve anlaşılmaz hale gelebilir.

Kaliteli Kodlama

∞ Yazılım Kütüphanesi Çoğu Zaman Sizden Daha Hızlıdır

- Yazılımcılarda kendine güvenden kaynaklanan kendi fonksiyonunu kendi yazmaya yönelik bir eğilim vardır.
- İşimize uygun bir kütüphane fonksiyonu varsa (ve özellikle de bu fonksiyon derleyicinin kütüphanesinde ise) bizim benzer fonksiyonlarımızdan neredeyse her zaman daha hızlı çalıştığı gözlenmiştir.
- Bunun sebebi derleyicinin kütüphanesindeki fonksiyonların binlerce kez test edilmiş ve çok kez optimize edilmiş olmasıdır.
- Tavsiye edilen eğer kaynak kodun sizde bulunması ve güvenilir olması gibi problemleriniz yok ise kütüphane fonksiyonlarının kullanılmasıdır.

Kaliteli Kodlama

🌀 Optimizasyonu Sona Bırakın

- Kodun daha hızlı, daha verimli çalışması için, hafızada daha az yer kaplaması için veya buna benzer çeşitli sebeplerle kodu optimize eder yani kısaltır ve kesip biçeriz.
- Ancak lütfen optimizasyonu, kod yazarken değil, kod yazma işlemini bitirdikten sonra, hatta testinizi yapıp kodunuzun sağlıklı çalıştığını doğruladıktan sonra yapın.
- Yanlış anlaşılmaya yol açmayalım; Kodu gelişigüzel, paçavra gibi yazıp, düzeltmeyi sonraya bırakın anlamında söylemiyoruz. Tabii ki her zaman kodunuzu çok temiz, düzenli ve mümkün olduğunca kısa ve basit yazmalısınız. Kastedilen optimizasyon amaçlı çabaları en sona bırakmanız gerektiridir.
- Bu tavsiyenin birçok sebebi var. En basit sebep çoğu zaman optimizasyon kodu karmaşıktır ve üstelik çoğu zaman kodun sadece çok kritik ve ufak bir bölümünde yapılması gereklidir.
- Bu sebeple kod yazmayı ve testini bitirdikten sonra sadece hızlı veya verimli çalışması gereken yerlerde optimizasyon yapmak daha akıllıca olacaktır.
- Optimizasyon yapmanın da bir süre maliyeti olduğunu unutmayınız.

Kaliteli Kodlama

∞ İyi Tasarım Optimizasyona Gerek Bırakmaz

- Optimizasyon yapan bir derleyici kullanmak veya kaynak kodu kodlayıcının optimize etmesi projelerde sık rastlanan bir durumdur. Optimizasyonun avantajları olduğu kadar dezavantajları da vardır. Tabii ki optimizasyon bazı durumlarda gerçekten hayat kurtarıcı olabilir ancak dezavantajları şunlardır:
 - Kodun bakım yapılabilirliğini azaltır, aşırı optimizasyon tekniği uygulanmış kodu anlamak genelde çok zordur,
 - Derleyici çok kaliteli değilse veya biz optimizasyonun yan etkilerini tam bilmiyorsak koda tespit edilmesi çok güç hatalar sokabilir, ayrıca kodda optimizasyon yapılmazken uyur durumda bulunan yani ortaya çıkmayan, zarar vermeyen hatalar optimizasyon yapıldığında birden ortaya çıkabilir.
 - Optimizasyon ancak belli durumlarda uygulanabilir bir yöntem olduğu için her zaman kullanılacak metodik bir teknik olarak benimsenemez, bir projede optimizasyonla işinizi rahatça halledebilirken diğer bir projede bir çok sebepten dolayı optimizasyonu kullanamayacağınızı görebilirsiniz, optimizasyona bel bağlamak iyi bir şey değildir.

Kaliteli Kodlama

☞ İyi Tasarım Optimizasyona Gerek Bırakmaz (devam)

- O halde ne yapmalıyız? Nasıl bir tasarım yapmalıyız ki optimizasyona ihtiyaç duymayalım? Optimizasyona ihtiyaç duyduğumuz sebepleri listeleyelim ve olası çözümleri de verelim:
 - Kaybolan depolama alanı (Bellek) veya kalıcı depolama alanı (disk) yetersizliği durumu: Uzun zamandır tespit edilmiş olan bir gerçek var ki donanım hemen her zaman yazılımdan daha ucuzdur ve her geçen gün daha da ucuzlamaktadır. Bellek veya diğer depolama alanlarının maliyeti birçok yazılım ekibinin işgücü maliyetinden ve gecikmiş bir projenin ceza ödemelerinden her zaman daha ucuzdur.
 - Mikroişlemci/Mikrodenetleyici hızının yetersizliği durumu: Bunun için daha hızlı bir mikro- işlemci/mikrodenetleyici kullanmanızı önerebiliriz. Ancak asıl önerimiz sadece yüksek hız gerektiren kod bölgelerine manuel veya derleyicinin optimizasyonunu uygulamanızdır. Genelde bir yazılımda hız sorunu yaratan bölgeler çok sınırlıdır. Kodu, kod kısımlarının işleyiş zamanları açısından incelemeli ve gerçekten zaman alıcı ve hızlı çalışması gereken kısımları tespit etmeli ve bu kısımları optimize etmelisiniz. Tüm bir kodu optimize etmek zorunda kalıyorsanız lütfen tasarımınızı yeniden gözden geçirin.

Kaliteli Kodlama

☞ Sürekli Düzenleme, Sürekli Temizlik ve Toplam Kalite

- The Pragmatic Programmer isimli kitapta yazar Kırık Camlar Teorisinden bahseder. Biraz uyarlayarak özetleyecek olursak, eski bir bina düşünün, bahçeli bir köşk, sahipleri bir süreliğine bir yere gitmiş olsun. Bu bina bir süre hiç dokunulmadan kalır. Ta ki ilk penceresi kırılana kadar. İlk kırık pencere hemen onarılsa bina yine bir süre daha idare eder. Ancak ilk kırılan pencere bir hafta içerisinde onarılmaz ise kısa zamanda içine hırsızlar, mahallenin top oynayan çocukları, eskiciler, tinerciler doluşuverir. İşte günümüzde her yerde duymaya alıştığımız toplam kalite ilkesi de buna benzer bir şeydir. Her birimiyle tıkr tıkr giden bir işletme başarıya koşar, ancak en önemsiz gördüğünüz bir bölümde bir şeyler aksayınca bu diğer işlere hızla yansır.
- Kendi kodunuza dönecek olursak kodunuzun bir parçasını düzeltirken hem işinizi yapın hem de kodda tertip ve düzen sağlayın. Bu refactoring olarak adlandırılır. Gereksiz derece uzun yazılmış işlevleri kısaltın; kod tekrarını azaltmak için işlevler kullanın; karmaşık algoritmaları basitleştirin; değişken isimlerini anlamlılaştırın; girinti çıkıntıları kod standartlarına uygun hale getirerek okunaklılığı sağlayın; fazla boşlukları alın; yorumlar ekleyin veya güncelleyin; kullanılmayan ve gerçekten gereksiz olduğunu bildiğiniz değişkenleri silin. Buna benzer birçok işlem daha yapın. Daha sonra yaparım demeyin, muhtemelen yapmaya zaman bulamayacaksınız ve testlerden sonra koda dokunmaya korkacaksınız. Emin olun bu düzen birçok kod hatasını daha yazarken görmenizi sağlayacak ve ileride hataların bulunmasını çok kolaylaştıracak.

Kaliteli Kodlama

∞ Sürekli Düzenleme, Sürekli Temizlik ve Toplam Kalite (devam)

- Projenizde kodlamaya başladıktan sonra testlere kadar sürekli iyileştirme (refactoring) yapabilirsiniz ve yapmalısınız. Optimizasyonu ise proje sonuna doğru yapınız. Refactoring optimizasyon ile aynı şey değildir. Farklarını listeyecek olursak:
 - Optimizasyon mükemmele yakın veya ulaşılabilecek en iyi noktaya yakın olmak için yapılır,
 - Refactoring daha iyi bir noktaya ulaşmak için yapılır,
 - Optimizasyonun kodun sadece gerekli yerlerinde yapılması tavsiye edilir,
 - Refactoring kodun her yerinde yapılabilir,
 - Optimizasyon hemen sonuç almak için yapılır, size kazanç sağlar,
 - Refactoring ise iyilik yapmak gibidir, hemen sonuç vermez ama bir gün mutlaka size geri döner,
 - Optimizasyon sonunda kodunuz daha okunaksız olabilir,
 - Refactoring'in amaçlarından biri kodun daha okunaklı olmasıdır.
 - Kodunuzu geliştirdikçe ve yeni özellikler ekledikçe kodun karmaşıklık seviyesi artar. İlk başta çok kolay ve bakımı yapılabilen kodlar bir zaman sonra hangi parçasının ne iş yaptığını belirleyemeyeceğiniz dev kod yığınları haline gelebilir. Bunu engellemenin tek yolu da sürekli basitleştirici ve düzenleyici türden refactoring yapmaktır. Yani bir yandan yeni eklemelerle kod karmaşıklaştıkça siz de öte yandan refactoring ile onu sürekli basitleştirmeli, sadeleştirmelisiniz.

Kaliteli Kodlama

∞ Büyük Diziler Tanımlamak

- Bazen çok büyük bir dizi tanımlamak zorunda kalabilirsiniz. Örneğin birçok kayıdı hafızada tutup çabucak erişmek için buna ihtiyaç duyabilirsiniz.
- Çok büyük bir dizi tanımlayacaksanız bunu stack'te değil heap'te yapın.
- Her derleyici ve işletim sistemi genellikle bir uygulama için stack büyüklüğünü sınırlı tutar ki bu da tahmini olarak lokal değişkenlerinizin tutabileceği yer kadardır. Derleyici stack büyüklüğünü ayarlarken çok büyük dizileri Stack'te tutacağınızı genellikle öngörmemiştir. Bu durumda Stack aşımı hatasına düşmemek için dizilerinizi heap'te yaratmanızı tavsiye ederiz.
- Örneğin bu işlem C dilinde malloc ve C++ dilinde new sözcüğü ile yapılabilir.

Kaliteli Kodlama

∞ Acaba Rastgele Sayılar Ne Kadar Rastgele?

- Rastgele sayıların sahip olması gereken iki ana özelliği vardır, diğer deyişle şu an bizi ilgilendiren iki ana özellik olmalı. Bunlar şunlardır:
 - Tahmin edilmesi zor olmalı, yani gerçekten rastgele olmalı. (Unpredictability)
 - Eşit dağıtılmış olmalı, yani herhangi bir sayının çıkma ihtimali sabit olmalı. (Uniform distribution)
- Şimdi bilgisayarları göz önüne alacak olursak bilgisayarlar ile rastgele sayılar üretmek gerçekten çok çok zor. Neden mi? Çünkü bilgisayar prensipte öngörülebilir (deterministik) çalışsın diye üretilmiş bir makine, hatta hiç rastgele çalışmasın, hep tutarlı çalışsın diye tasarlanmış bir makine. Bu olay örnek verecek olursak sadece kırmızı, mavi, sarı ve yeşil ana renklerini bilen bir ressamın pembe gül çizdirmeye çalışmak gibi birşey.
- Bilgisayarla rastgele sayı üretmeye çalıştığımızda önümüzdeki seçenekler şunlar:
 - CPU saatinin veya kristalinin “tik-tak”ını kullanmak
 - Önceden hazırlanmış (pseudo-random) rastgele sayı listesini veya algoritmasını kullanmak.
- Her iki yöntemin de başta belirttiğimiz rastgele sayıların iki ana özelliğini sağlamada zaafları vardır.

Kaliteli Kodlama

- ☞ Acaba Rastgele Sayılar Ne Kadar Rastgele? (devam)
 - Sonuçta kısa sürelerde hızlı hızlı rastgele sayı üretilince CPU “tik-tak”ından yararlanmak verimli olmuyor ve uzun sürelerde de algoritma veya listeler verimli olmuyor.
 - Bu sebeple günümüzdeki bilgisayarlarda genellikle iki basamaktan oluşan hibrid bir çözüm kullanılır.
 - Önce ana gereklilik olan “rastgele”liği sağlamak için CPU “tik-tak”ından faydalanarak bir başlangıç sayısı (seed) bulunur.
 - Sonra ana gereklilik olan “eşit dağıtık”lığı sağlamak için bu az önceki başlangıç sayısı indeks veya girdi olarak kullanılır ve eşit dağıtılmış çok uzun bir üsteden veya daha iyisi bir algorithmadan istenilen rastgele sayı alınır.



Kaliteli Kodlama

- ☞ Kayan Noktalı Sayılar (Floating Point Numbers) Gerçek Baş Belaları
 - Çeşitli dillerde single veya float gibi anahtar sözcüklerle temsil edilen tek duyarlıklı kayan noktalı sayılar günümüzde neredeyse kullanım dışı kalmış, sorun çıkaran özelliklerdir.
 - Bu türden değişkenlere atanan sayılar büyüdükçe veya küçüldükçe duyarlılıkları azalır. Örneğin kayan noktalı olarak tanımlanmış olan 9000000 ve 1.5 sayılarını toplayınca 9000001.5 yerine 9000002 buluruz. Bu kadar küçük sayılarda bile kayan noktalı sayılarda yuvarlama yapılmaktadır.
 - Mühendislik hesaplarında çift duyarlıklı kayan noktalı sayılar (örneğin C'de double) kullanılarak bayağı bir duyarlılık elde edilebilir. Ancak her bir kuruşun önemli olduğu bankacılık işlemlerinde böyle bir hata bile kabul edilebilir değildir.
 - Bankacılık işlemlerinde sayıları oluşturan rakamları BCD (Binary Coded Decimal Number-İkili Kodlu Ondalık Sayı) olarak tutmak iyi bir yöntemdir.
 - Diğer bir yöntem de liralari küsuratlı tutmak yerine birim olarak kuruş kullanıp tipini tamsayı olarak belirleyerek işlemleri yapmaktır.

Kaliteli Kodlama

☞ Kayan Noktalı (Float) Değerleri Eşitlik (Eşitsizlik) Karşılaştırmasında Kullanmayın

- Kayan noktalı (float) türden değişkenler iç özellikleri sebebi ile kendisine atadığınız değeri tam olarak tutamazlar ve bu değere en yakın bir değeri içlerinde tutarlar.
- Bu sebeple float türünden değişkenleri karşılaştırmalarda kullanırsanız yanlış sonuçlar elde edersiniz.
- Bu türden değişkenlerin ancak hemen hemen birbirlerine eşit olduğunu tespit edebilirsiniz.
- Bu durumda değişkenlerin birbirlerinden farklarının mutlak değerinin bir epsilon değerinden küçük olduğunu kontrol etmeniz yeterlidir. Epsilon matematikte küçük pozitif bir niceliği göstermek için kullanılan semboldür.
- Örnek: Kayan noktalı değişkenlerin karşılaştırmalarda kullanımları.

```
float a; float b;
```

```
if ( a == b ) { bir kod parçası }           // Yanlış kullanım
```

```
float epsilon = 0.001;
```

```
if ( fabs ( a - b ) < epsilon ) { kod parçası } // Doğru kullanım
```

Kaliteli Kodlama

- ✎ Programlama Dilinizin Sağladığı, İstisnai Durum Ele Alma (Exception Handling) Özelliğini, Kod Akışını Sağlamak İçin Kullanmayın
 - Birçok programlama dili istisnai durumlarda kontrolü sağlamak için programcıya kolaylıklar ve mekanizmalar sağlar (exception handling). Örneğin sıfıra bölme hatası oluştuğunda, bellek aşımı olduğunda istisnai durum oluşturulur.
 - Yeni başlayan birçok programcı bu kolaylığı bilmeden suistimal eder ve istisnai durum deyimlerini kod akış anahtar sözcükleri gibi kullanır. Örneğin diskteki bir dosya açılmadığında normalde fonksiyonun dönüş değerini incelemeniz gereklidir ve buna göre akışınızı yönlendirmelisiniz.
 - Oysa ki tecrübesiz programcılar dosya açma işlevini bir istisnai durum bloğu içine koyarlar ve nasıl olsa dosya bulunamazsa istisnai durum oluşur ve kodum da kilitlenmeden yoluna devam eder diye düşünürler.
 - Oysa ki birçok başka sebeple dosya açılamamış olabilir. İşletim sistemi istisnai durum vermeyebilir, vb.

Kaliteli Kodlama

- Öncelikle yanlış örneği sembolik kod olarak verelim:

```
try
{
    dosya_isaretcisi= Dosya.DosyaAc("Dosya_Adi");
}
catch (Exception e)
{
    // Olağanüstü bir durum
}
```

- Şimdi de doğru hazırlanmış örnek. Öncelikle dosyanın varlığı kontrol ediliyor. İstisnai durum ise gerçekten çok istisnai bir durum için kullanılıyor:

```
try
{
    int sonuc= Dosya.DosyaVarmi("Dosya_Adi");
    if (sonuc==0) dosya_isaretcisi = Dosya.DosyaAc("Dosya_Adi");
    else
        DosyaBulunamadiYaz();
}
catch (Exception e)
{
    // Olağanüstü bir durum
}
```

Kaliteli Kodlama

∞ Klasik Hata Kalıpları (Anti-Pattern'ler)

- Klasik hata kalıpları programları kodlarken veya tasarlarken, özellikle de deneyimsiz yazılımcıların düştüğü, ancak en deneyimli yazılımcının bile düşebileceği klasik hatalardır.
- Bu hata kalıplarından kastımız kodda bulunabilecek bug'lar değil fakat uygulamalardaki, tasarımdaki veya yazılım yönetimindeki yöntem hatalarıdır.
- Klasik bir hata kalıbının yapıldığı bir yazılım, “bakım zorluğu, taşınma zorluğu, tekrar kullanım zorluğu, genişlemeye kapalılık, maliyet yüksekliği” gibi, yazılımda istenmeyen kötü özelliklerden bir veya birkaçına bürünür.

Kaliteli Kodlama

☞ Klasik Hata Kalıpları (Anti-Pattern'ler) (devam)

- Kopyala/Yapıştır Hata Kalıbı - Kopyala yapıştır hata kalıbı işlevler içinde durabilecek kodlar yerine sürekli bir yerden kopyalanıp diğer yerlerde kullanılan kodlar kullanma alışkanlığına denir. Yeni başlayan programcılar varolan kodlara dokunmaktan çekindikleri için böyle bir yöntemle başvururlar. Tecrübeli programcılar ise işlev içine koymayı ihmal ettikleri birçok hazır kodları bulunduğu için böyle bir yöntemle başvururlar. Bu stilin dezavantajı bir problem çıktığında bütün kopyalanan yerlerde aynı hatanın düzeltilmesi gerekliliğidir.
- Anaç Nesne (God Object) - Büyük bir ana sınıf ve içerisinde birçok işlev ve üye değişken var ise, bu durum ilgili sınıfın yeterince bölünemediği anlamına gelir. Genellikle tasarımda sorunlar olduğuna işaret eder.
- Tasarım Deseni Çöplüğü - Bu programlama stilinde programcı her iş için bir tasarım deseni kullanır. En basit işler için bile bir tasarım deseni kullanıldığı için bir süre sonra programın anlaşılabilirliği iyice kaybolur. Gerçekten ana işin yapıldığı satırlar ile tasarım desenleri için kullanılan satırları ayırt etmek imkânsız hale gelir.

Kaliteli Kodlama

🌀 Klasik Hata Kalıpları (Anti-Pattern'ler) (devam)

- En Az Çaba İle Programlama Gafleti - Bu programlama tipinde programcı mümkün olan en az çaba ile programlama yapar. En sık rastlanan örneği bir programcının programda bir x işaretçisi null olduğu için program çıktığında oraya sadece “if (x! = null)” şeklinde bir günü kurtarma çözümü koymasidir. Bu programlama stili problemlerin gerçek sebebin araştırılmayıp sadece günü kurtarma çabası ile birkaç satır eklenerek geçiştirilmesi şeklinde özetlenebilir.
- Türet Türet Dur Hata Kalıbı (BaseBean) - Nesne yönelimli programlamada sık yapılan bu klasik hata kalıbında kodun içerisi türetilmiş sınıflarla doludur. Bir sınıf bir kaç kez üst üste türetilmiştir. Oysa her gerekli olan türetmeler kodu kolaylaştırırken tam tersine gereksiz türetmeler kodun kullanılabilirliğini bir hayli azaltırlar. Gereksiz türetme işleminden kaçınmanın yolları vardır. Eğer uygun ise türetme özelliği yerine sahip olma özelliğini kullanmak çoğu zaman hayat kurtarır. Şöyle ki B sınıfı bir A'dır yerine B sınıfı bir A'ya sahiptir (veya Ayı kullanır) şeklinde tasarlanacak uygulama hem daha kolay hem de daha anlaşılır olur.
- Komite Kararı İle Tasarım - Birçok grup veya kişi bir araya gelir ve toplantılarla tasarımı yapar. Ya herkesin bir parçasına katkıda bulunduğu ama bir şeye benzemeyen garip bir tasarım ortaya çıkar ya da komitedeki en güçlülerin fi-kirlerine göre şekillenmiş ancak teknik olarak iyice incelenmemiş bir tasarım ortaya çıkar. Birinci türünde tarafların her biri kendi vizyonunun bir parçasını son ürüne sokabildiği için ortaya hiç kimsenin beğenmediği ama düzeltilmesi için de kimsenin tartışacak enerjisinin kalmadığı bir ucube çıkar. İkinci türünde ise komitedeki en güçlü lobinin vizyonuna göre şekillenmiş bir ürün ortaya çıkar ancak birçok teknik aksaklıkları olduğu için sonuç hiç kimseyi memnun etmez. Üstelik sonuçta komitedeki güçlü olan lobi mükemmel bir ürün çıktığını iddia eder. Olan kullanıcıya olur.

Kaliteli Kodlama

🌀 Neden Yazılım Hataları Vardır?

- İnsan hatası: Bu konuda yapabilecek şeyler elbette var ancak ne kadar önlem alınırsa alınsın insanlar hata yapacağı için bu tür hataları sıfıra indirmek mümkün görünmüyor. Bu hatalar gerek yönetici gerekse yazılımcı tarafından yapılıyor olabilir.
- İletişim sorunları: Organizasyondaki iletişim eksikliği veya iletişimin hiç olmaması yazılımlardaki hataları bir hayli arttırır. (Hower 2012)
- Karmaşık yazılım: Yazılım gereksiz yere karmaşıklaştıkça hatalar artar. (Hower 2012)
- Sürekli değişen gereksinimler: Gereksinimlerin sürekli değişmesi kodlama-test döngüsünde tutarsızlıklara yol açar ve hatalar artar. (Hower 2012)
- Anlaşılmayan, yarım veya belirsiz olan gereksinimler: Gereksinimler anlaşılır yazılmadığı zaman hata oranı artar.
- Zaman baskısı: Gerçekleştirme için ayrılan süre kısaltıldıkça hata oranı artar. (Hower 2012)
- Dokümantasyon eksikliği veya hataları: Kod içerisindeki yorumlar veya kodu anlatan dokümanlara yeterince önem verilmediğinde yazılım hataları artar. (Hower 2012)
- Ego: Proje ekibindeki insanların egosu büyüdükçe koddaki hatalar artar. (Hower 2012)

Kaliteli Kodlama

∞ Neden Yazılım Hataları Vardır? (devam)

- Kalitesiz yazılım geliştirme araçları: Yazılım geliştirme araçlarının kalitesi arttıkça koddaki hata oranı düşer. Örneğin iyi bir IDE, editör veya hata ayıklayıcı kullanıldığında hata yapma olasılığı azalır.
- Yetersiz, uygun olmayan donanım: Donanım yetersizse veya uygun değilse yazılımda hata çıkma olasılığı artar.
- Eğitim azlığı: Personel eğitimsiz ise hata olasılığı artar.
- Kötü tasarım: Tasarım uygunsuz ise hata olasılığı artar. Genelde eğitim azlığından kaynaklanır.
- Yanlış kullanılan teknoloji: Teknoloji yanlış kullanılırsa hata olasılığı artar. Genelde eğitim azlığından kaynaklanır.
- Yetersiz test: Gerek zaman baskısı gerekse ihmalden dolayı yetersiz test yapılması koddaki hata oranının yüksek çıkmasına sebep olur.
- Kötü kodlama alışkanlıkları: Bu tür alışkanlıklar da koddaki hata oranını arttırır.
- İhmalkarlık: İhmalkarlık da önemli bir hata sebebidir.
- Fiziksel dünyanın teorik dünyaya uyuşmaması: Fiziksel dünyanın teorik olarak hesaplanandan farklı davranması ve bunun zamanında tespit edilememesi de örneğin iletişim hatlarındaki sorunların başında gelir.

Kaliteli Kodlama

🌀 Yazılıma Yeni Başlayanların Sık Yaptıkları Hatalar

- Yazılıma yeni başlayanların en sık yaptıkları hataları aşağıda sayabiliriz:
 - İlkendirilmemiş değişken
 - Tampon bellek taşması
 - Saçma yeri gösteren işaretçi (C ve benzeri işaretçi tanımlı diller)
 - Görevlerin kilitlenmesi - Deadlock (multi-task programlarda)
 - Sonsuz döngü
 - Değişken taşması
 - Bellek kayıtları bozulması (Memory Corruption)
 - Belleğin boşlukta ve kullanım dışı kalması (Memory Leak)
 - Döngü sayımı bir yanlış (Sıfır yerine birden saymaya başlamak, n-1 yerine ne kadar saymak gibi)
 - Düzeltme yaparken yeni hata oluşturmak, Yazılım Gerilemesi (Regression)
 - Stack taşımı
 - Ölçü birimi hataları (Örneğin inch olan bir değeri cm imiş gibi kullanmak)
 - Tip çevrimlerinde hatalar
 - Kullanıcı girişi kontrolünde hatalar

Kaliteli Kodlama

∞ Çalıştırılabilir Kodum Bir Bilgisayarda Çalışıyor Diğer Bir Bilgisayarda Çalışmıyor

- Bu problemin birkaç sebebi olabilir, bunlara örnek vermek gerekirse:
 - İki bilgisayarın güvenlik ayarlarının farklı olması (diske yazma hakkı, şebeke güvenlik ayarları, vb),
 - Programın konfigürasyon dosyalarının eksik olması,
 - Kodun çalışması için Java Virtual Machine veya .Net framework gerekiyorsa bunların eksik olması,
 - Dinamik kütüphanelerin eksik olması (Windows için .dll uzantılı dosyaların eksik olması)